

OperationKivotos

기능별 · 유닛별 런타임 흐름 도식

Addressable 비동기 리소스 파이프라인 · 오브젝트 풀링 · 데이터 주도 설계 · 행동트리 기반 몬스터 AI

1. 시스템 아키텍처 (Managers 서비스 로케이터)

2. 부팅 & 초기화 시퀀스

3. 씬 전환 흐름

4. 데이터 파이프라인 (Excel→SO/JSON→DataManager)

5. 리소스 & 풀링 파이프라인 (Load→Instantiate→Destroy)

6. 섹터 진입 → 몬스터 스폰 흐름

7. 몬스터 생명주기 & 풀 재사용

8. 행동트리(BT) 의사결정 흐름

9. 전투 & 데미지 흐름 (공격→투사체→피격→HP)

10. HP바 생명주기 (월드 추적 UI)

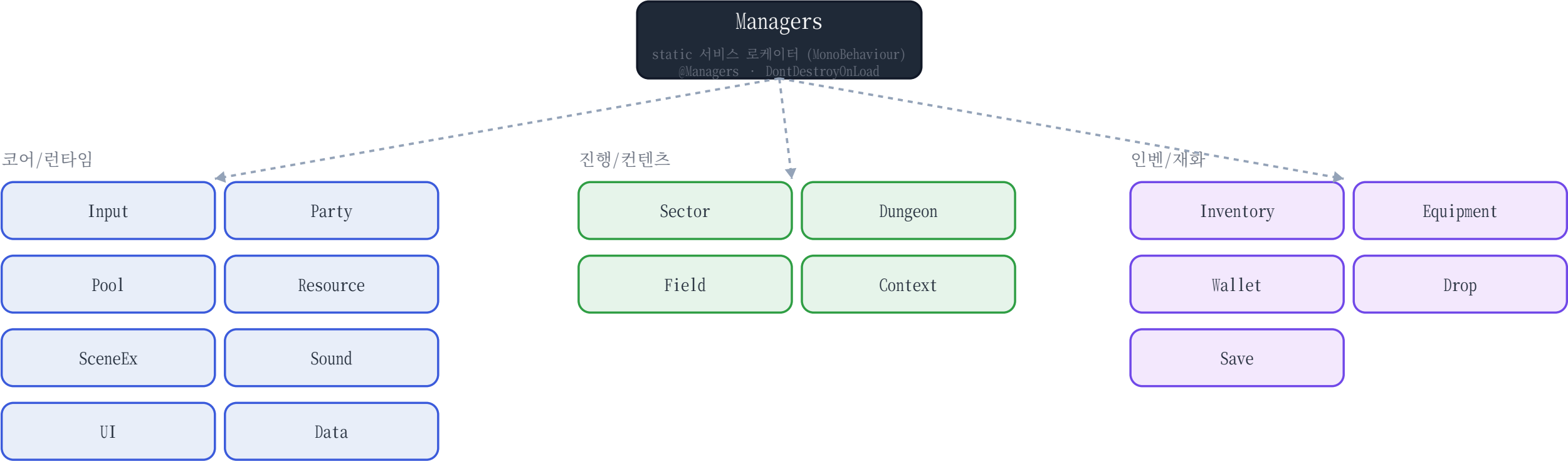
11. 파티/플레이어 & 입력 루프

12. 던전 클리어/실패 판정 흐름

13. 세이브/로드 흐름 (AES + 원자적 쓰기)

1 · 시스템 아키텍처

정적 서비스 로케이터 Managers 를 통해 17개 매니저(POCO)에 전역 접근



코어 진행/컨텐츠 인벤/재화

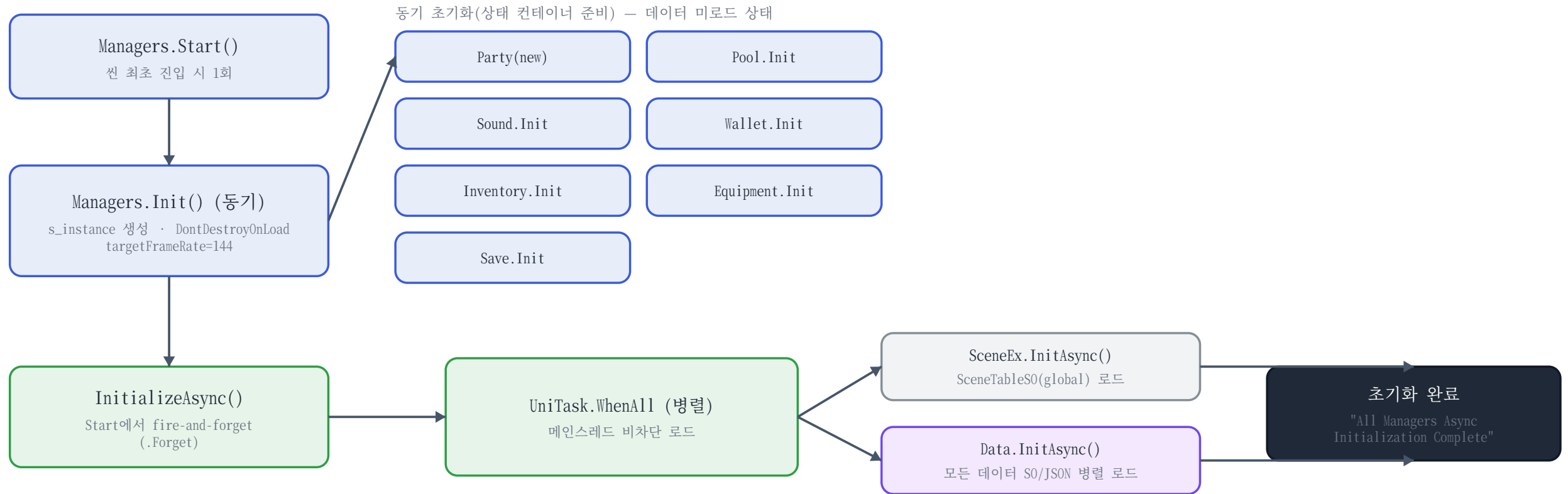
◆ 아키텍처 노트

장점: 어디서든 `Managers.X` 로 즉시 접근 → 보일러플레이트 최소화. 단점(실무): 의존성이 코드에 숨겨져(생성자·필드에 안 드러남) 결합도↑, 단위 테스트에서 목 주입 불가, 초기화 순서 결합. → 핵심 매니저 인터페이스화(`IResourceManager` 등) + 컨트롤러에 `Init(deps)` 주입 지점 확대 시 테스트 가능성↑. 전체 201파일 중 98파일(49%)이 `Managers.X` 참조 · `Managers.Resource` 단독 150회.

2 · 부팅 & 초기화 시퀀스

Managers.Start → 동기 Init() → 비동기 InitAsync() 병렬 로드

02/13

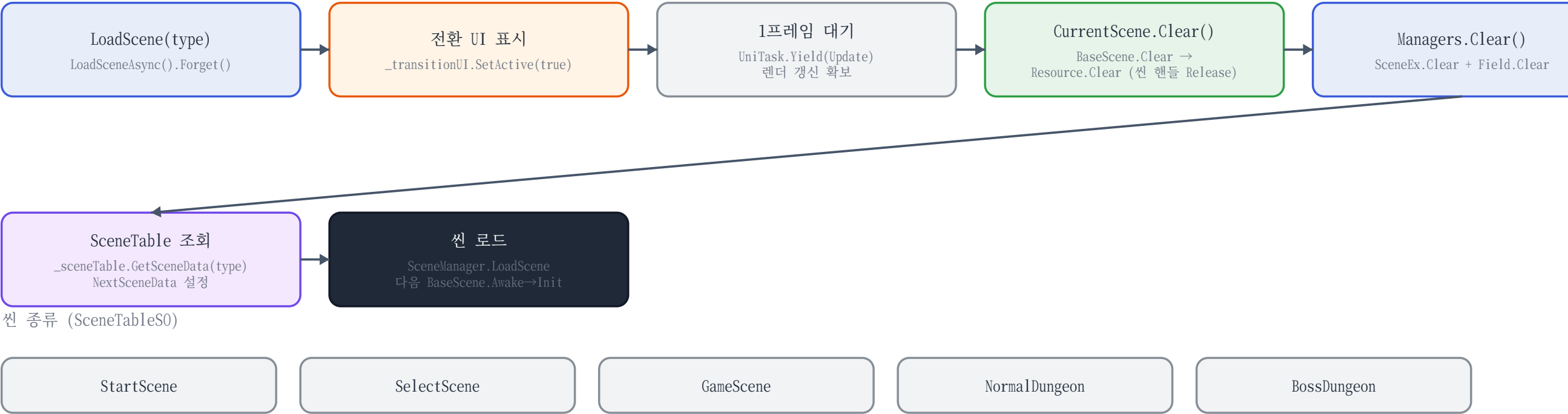


◆ 초기화 순서 안전성

동기 Init()은 상태 컨테이너만 준비하고, 실제 콘텐츠 데이터는 InitAsync에서 Addressable로 병렬 로드한다. InventoryManager.Init 등은 Data에 의존하지 않고 슬롯 구조만 생성하므로 초기화 순서 레이스는 없다(현재). 단, 매니저별 썬 스크립트 정리 계약이 제각각 → 공통 IManager{InitAsync, ClearScene} 도입 여지.

3 · 씬 전환 흐름

SceneManagerEx.LoadScene → 정리 → SceneTableSO 조회 → 다음 씬



◆ 메모리 수명 경계

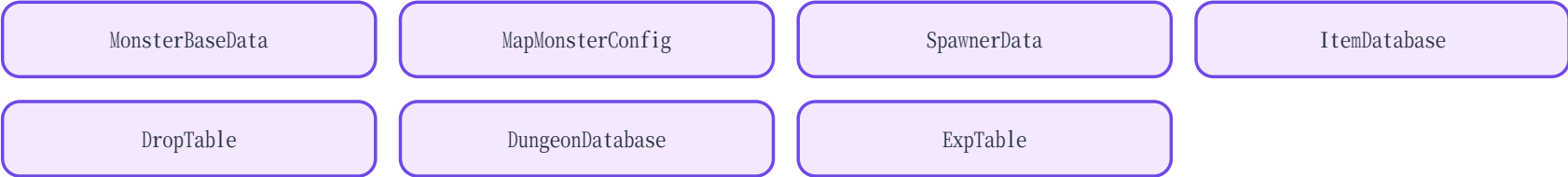
씬 전환의 핵심은 CurrentScene.Clear()에서 Resource.Clear로 '씬 핸들'만 Release하는 것 — 글로벌 핸들(플레이어 · 공용UI)은 유지. 이로써 씬 왕복 시 무거운 Addressable 에셋이 누적되지 않는다(Memory Profiler로 누수 0 검증). SceneManagerEx는 POCO라 코루틴 대신 UniTaskVoid로 씬 로드를 구동.

4 · 데이터 파이프라인

Excel(오프라인) → JSON/ScriptableObject → DataManager 만능 딕셔너리



소비 데이터 예



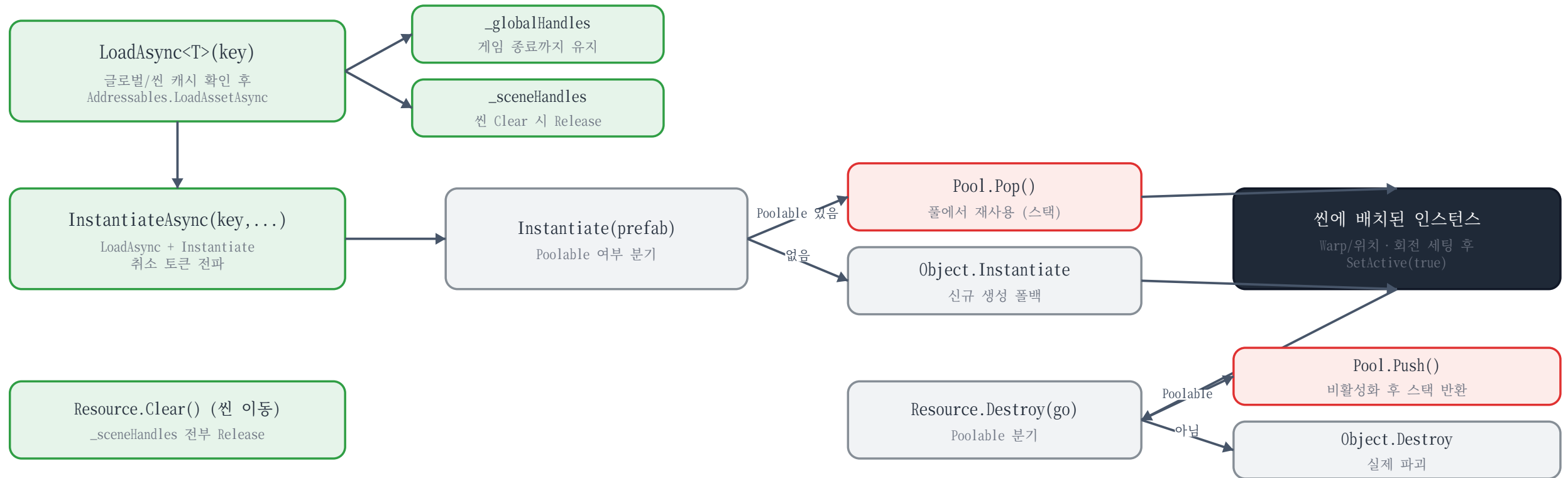
◆ 데이터 주도 · 확장성

데이터 추가 시 InitAsync에 한 줄만 등록. 타입을 키로 하는 단일 딕셔너리(_dataDicts)로 모든 테이블을 통합 → GetData<TKey, TValue>로 타입 안전 조회. SO는 각자 CacheData로 자기 딕셔너리를 바구니에 담고(IDataCacheable), JSON은 ILoader로 파싱 — 확장 시 로더/캐서만 추가.

5 · 리소스 & 풀링 파이프라인

LoadAsync(핸들 버킷) → InstantiateAsync(로드+풀) → Destroy(풀 반환)

05/13



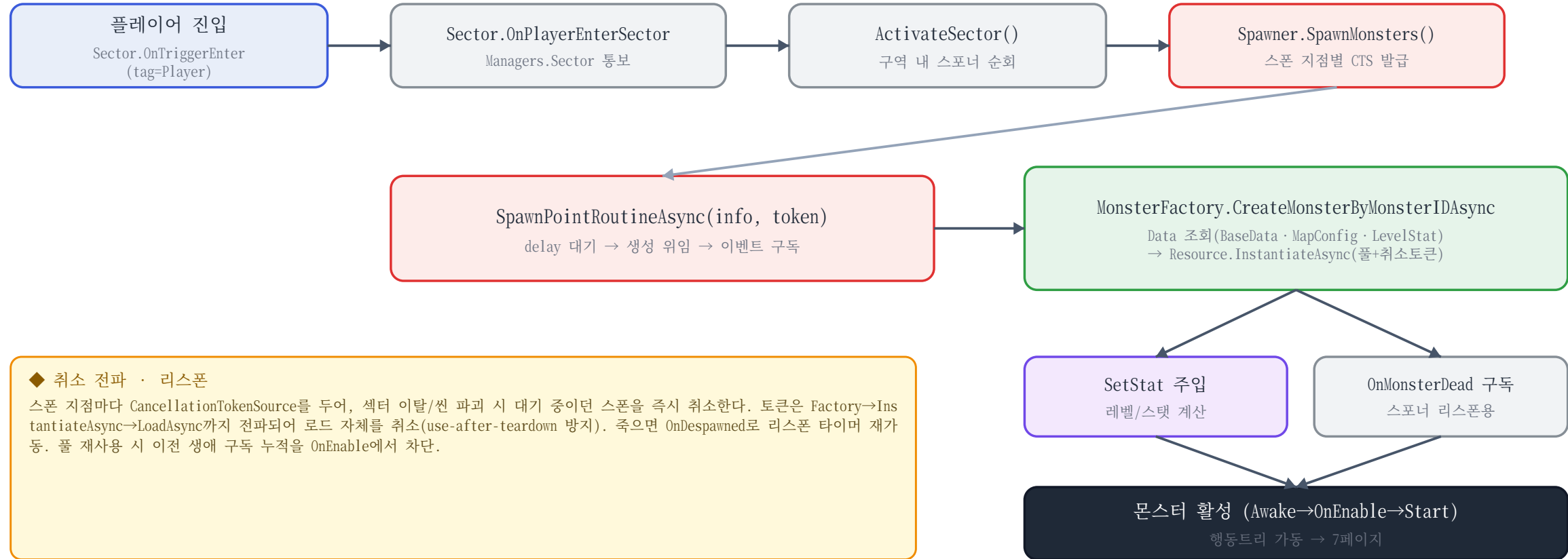
◆ 풀링 정합성

핵심: 글로벌/씬 핸들 '버킷 분리'로 메모리 수명을 명시적으로 관리. 스폰은 반드시 **Instantiate(풀 경유)**로 통일해야 풀이 실제로 재사용된다(과거 **async** 경로가 **Object.Instantiate**로 풀을 우회 → 스폰 스파이크 버그를 프로파일러로 특정·수정). 원샷 VFX는 **AutoReturnToPool**로 반환 경로를 보강(스폰만 하고 회수 안 하던 누수 차단).

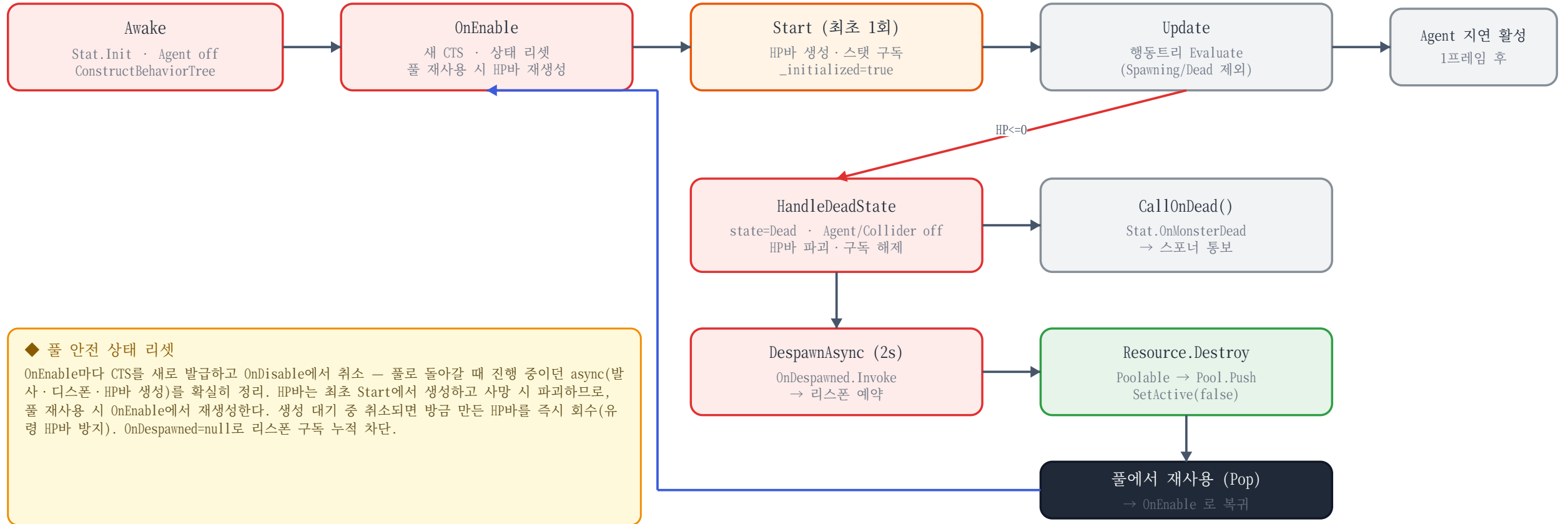
6 · 섹터 진입 → 몬스터 스폰

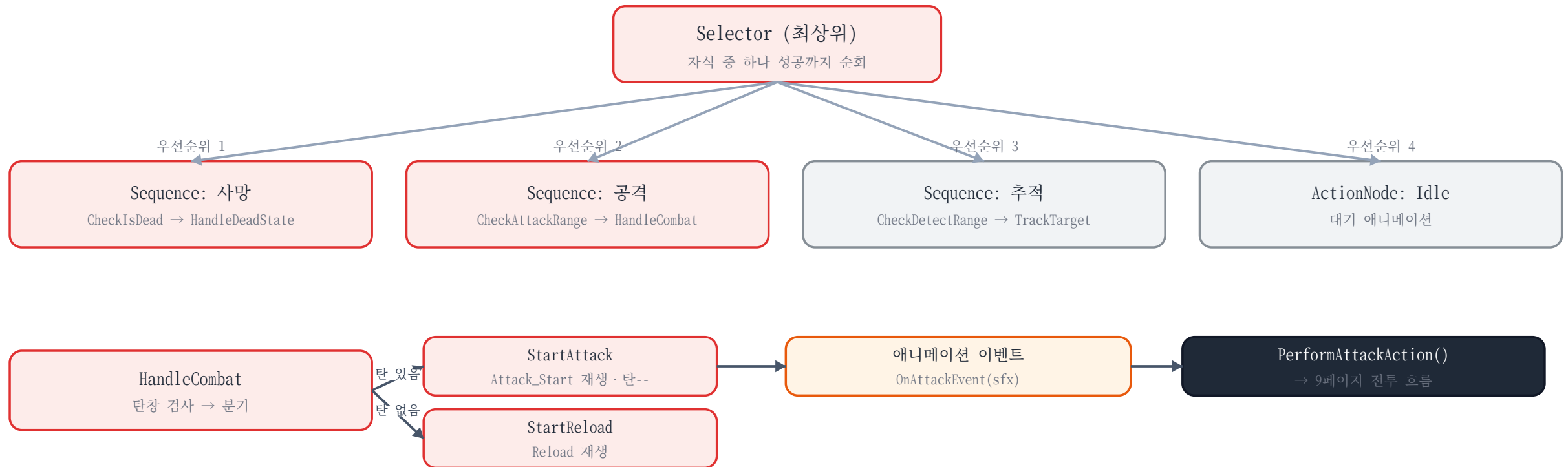
플레이어 트리거 → Sector 활성화 → Spawner 루틴 → Factory → 풀

06/13



7 · 몬스터 생명주기 & 풀 재사용





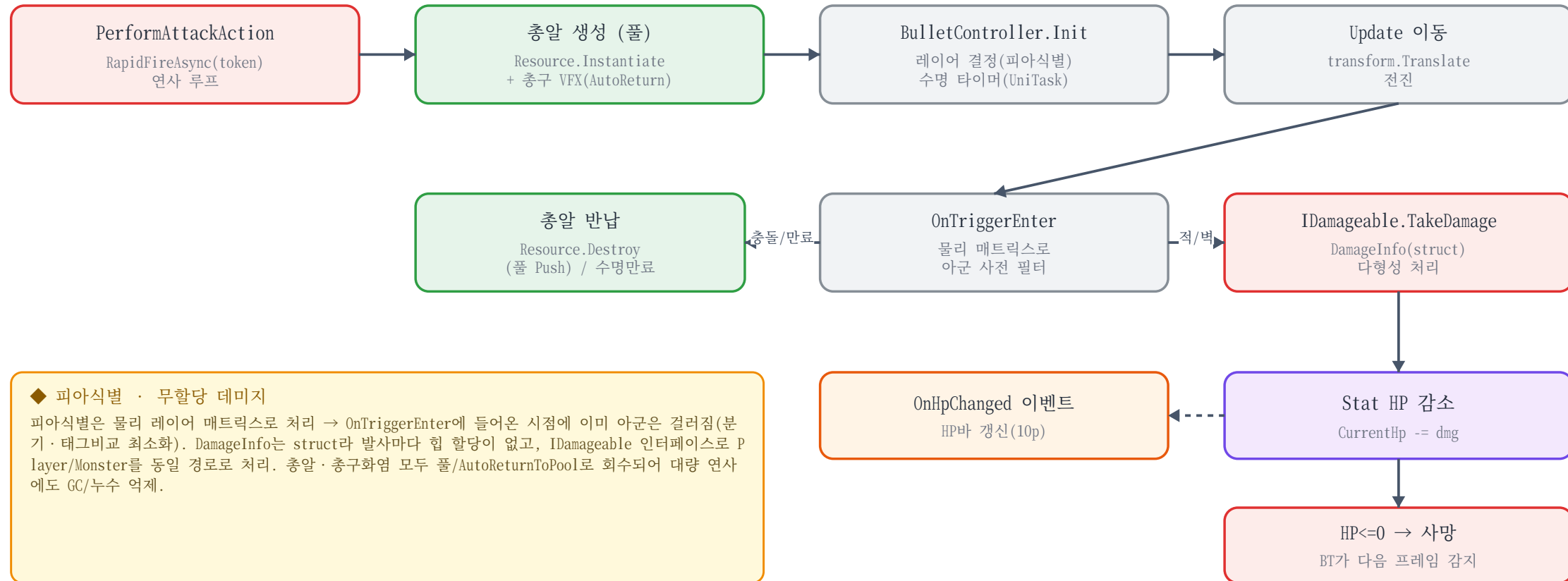
◆ 무할당 BT

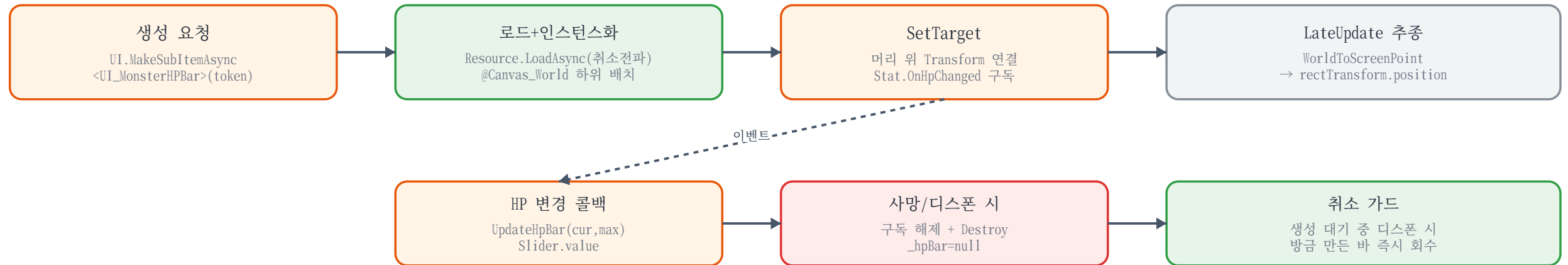
트리는 Awake에서 1회 구성되고 매 프레임 재사용된다. Selector/Sequence는 List<Node>를 foreach(구조체 열거자)로 순회하고 ActionNode는 캐싱된 델리게이트를 호출 → 프레임당 힙 할당 0. 상태(공격/장전 중)는 Running을 반환해 동작을 끝까지 유지. 실제 발사 타이밍은 애니메이션 이벤트에 위임(연출-로직 동기화).

9 · 전투 & 데미지 흐름

PerformAttackAction → 투사체(풀) → 충돌 → IDamageable → HP

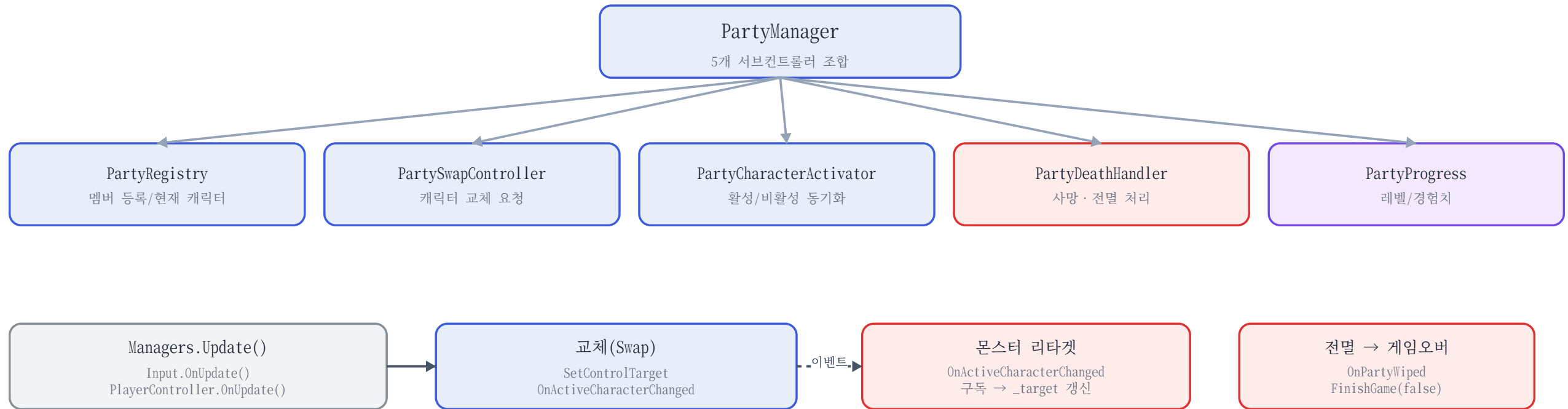
09/13





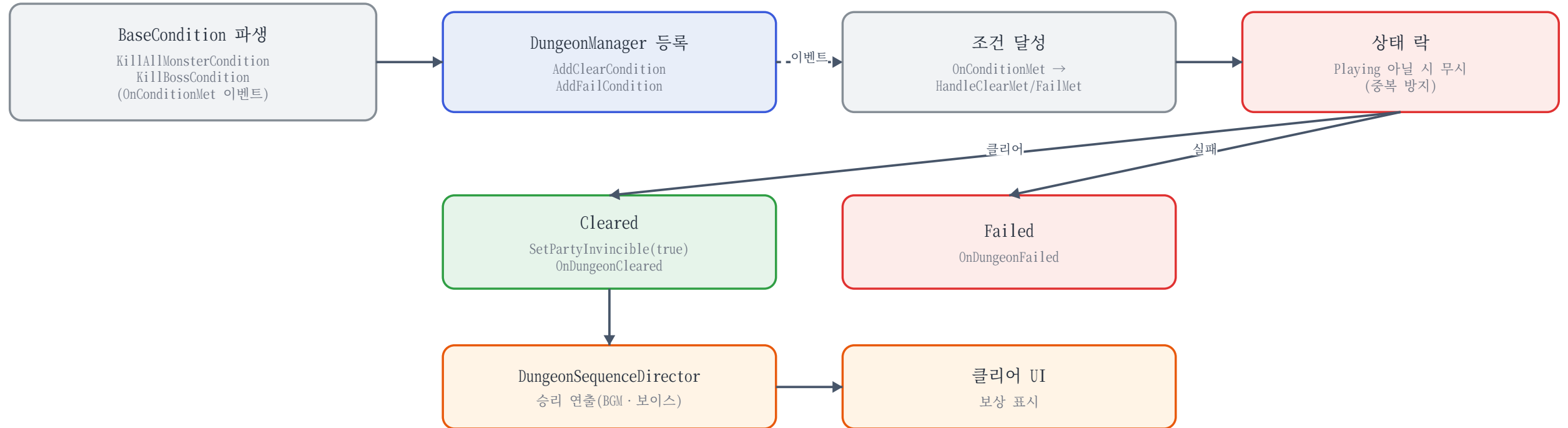
◆ 캔버스 격리 · 누수 이력

@Canvas_World는 월드 추적 UI 전용 '독립 캔버스'(sortingOrder 5)라 HP바의 매 프레임 이동이 메인 UI 캔버스 리빌드와 격리된다. 다만 모든 HP바가 한 캔버스에서 매 LateUpdate에 위치를 갱신 → 동시 몹이 많아지면 캔버스 배치 리빌드가 첫 병목(단일 섹터 설계에선 소규모라 무해). 과거 씬 왕복 시 HP바 누수(GameObject +291/왕복)를 추적 · 해결했고, 비동기 생성 취소 가드로 유령 HP바 재발을 차단.



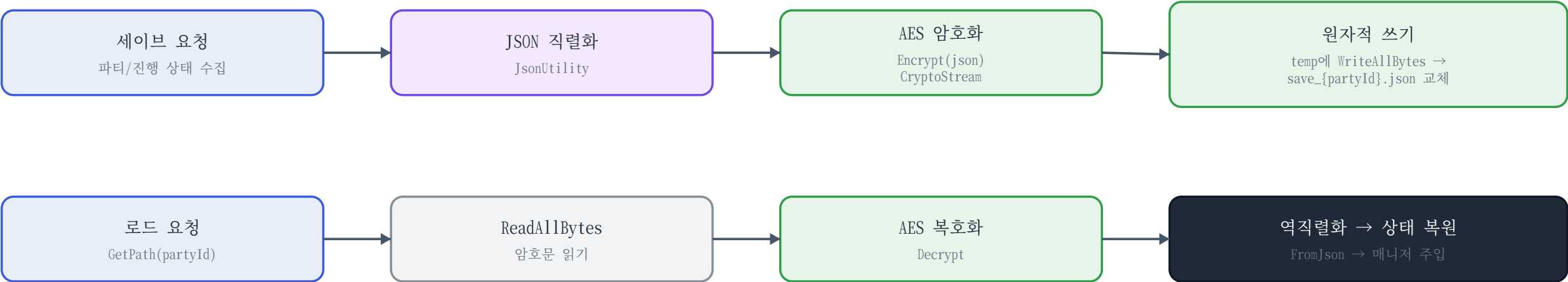
◆ 컴포지션 설계

PartyManager는 단일 거대 클래스 대신 5개 책임(등록 · 교체 · 활성화 · 사망 · 진행)을 서브컨트롤러로 분리한 조합(composition) 구조 — 이 프로젝트에서 가장 SRP를 잘 지킨 부분. 플레이어 입력은 Managers.Update에서 PlayerController.OnUpdate로 구동되고, 캐릭터 교체 시 OnActiveCharacterChanged 이벤트로 몬스터 타겟이 일괄 갱신된다.



◆ 조건 객체 · 상태 락

클리어/실패를 하드코딩 대신 '조건 객체' 리스트로 구성 → 던전마다 조건 조합만 바뀌 재사용(전략 패턴). DungeonManager는 상태(None/Playing/Cleared/Failed)를 락으로 관리해 중복 클리어 · 경합을 차단하고, 씬 전환 시 ClearDungeonData로 조건 구독을 해제해 누수를 막는다.



◆ 무결성 · 원자적 쓰기

세이브는 persistentDataPath에 파티 단위(save_{partyId}.json)로 저장하며, AES 암호화 + 임시파일 후 교체(원자적 쓰기)로 쓰기 도중 크래시에도 기존 세이브가 손상되지 않게 한다 — 이 프로젝트에서 가장 실무적으로 견고한 부분. 보강 여지: 세이브 버전 필드(스키마 마이그레이션) 및 손상 파일 폴백(백업본) 정책.